AI-Assisted Re-Architecture of Proprietary Software Suites into Open-Source Alternatives

An analysis of LLM-driven system analysis, the Model Context Protocol, reverse-engineering methodology, and GitHub as a coordination layer

1. Abstract and Introduction

Abstract

Software development is shifting from authoring code line by line toward directing AI systems that analyze, specify, and translate existing system behavior. This paper examines how large language models (LLMs), principally Claude, are used to study the observable behavior of proprietary application suites such as Adobe Creative Cloud and Microsoft Office, and to reconstruct their functionality as modular open-source software. It covers three technical threads: the use of long-context models for comprehension of interfaces, file formats, and rendering pipelines; the Model Context Protocol (MCP) as an integration layer that gives an LLM controlled access to codebases, logs, and execution environments; and the engineering methodology of interface-level reverse engineering. It then analyzes GitHub as the coordination infrastructure for these projects, and closes with technical, legal, and governance challenges. Section 4.5 examines ArtCraft, a suite of open-source, clean-room reimplementations of commercial creative applications published by the GitHub user storytold, as a case study, limited to what the project states publicly.

1.1 Paradigm shift

Traditional software engineering treats implementation as the bottleneck: a team reads requirements, designs modules, and writes code. For large application suites, the bottleneck was historically comprehension. A raster editor with layer compositing, non-destructive filters, color management, a plugin SDK, and a decades-old file format carries a specification that exists mostly as behavior, not as documents. Rebuilding such a system required years of manual observation.

LLMs change the cost structure of that comprehension step. A model with a context window of several hundred thousand tokens can ingest a file-format specification, a set of hex dumps, public SDK headers, and a test suite in a single session, then produce structured analysis: candidate field layouts, state-machine descriptions, and draft parsers. The developer's work moves from typing implementations to defining acceptance criteria, validating model output against ground truth, and curating the resulting specification.

1.2 Scope

This paper concerns **functional recreation**, not code reproduction. The material of interest is:

- UI and UX patterns (panel layout, tool semantics, keyboard conventions, document models).
- Rendering pipelines (compositing order, blend-mode mathematics, color-space handling).

- File-format parsers (PSD, AI, DOCX, XLSX, PPTX and their internal structures).
- Extension and plugin architectures.

The paper does not cover circumvention of license enforcement, redistribution of proprietary binaries, or extraction of proprietary source code. Those activities fall outside legitimate interoperability work and are discussed only as legal boundaries in Section 6.

2. The Role of Advanced LLMs in Software Deconstruction

2.1 Capabilities relevant to system analysis

Three properties of current frontier models matter for this work.

Long-context retention. Format analysis and architecture recovery depend on cross-referencing distant pieces of evidence: a field observed in a hex dump, a constant in a public header, and a behavior in a screenshot sequence. Large context windows allow these artifacts to coexist in one working set, which reduces the information loss that occurs when analysis is chunked across sessions.

Code comprehension across languages and abstraction levels. Models trained on large code corpora read C++, Rust, Python, TypeScript, and assembly listings with usable accuracy. They are effective at summarizing the control flow of an unfamiliar function, identifying standard algorithms (Huffman decoding, PackBits run-length encoding, DEFLATE, Porter-Duff compositing), and proposing names for recovered symbols. Accuracy degrades on heavily obfuscated or optimized code, and outputs require verification.

Structured output and tool use. Models can emit machine-checkable artifacts: JSON schemas, Kaitai Struct definitions, test vectors, and parser skeletons. Combined with tool use (Section 3), they can run those artifacts and iterate on failures.

2.2 Input classes

The inputs that developers supply to a model fall into several categories, each with different legal and technical characteristics.

Input class	Examples	Typical use
Public documentation	ISO/ECMA standards (ECMA-376 for Office Open XML), Adobe's published file-format notes, SDK reference	Baseline specification
Observed artifacts	Sample files, hex dumps, network captures, screenshots, recorded interaction traces	Behavioral specification
Disassembly or decompiler output	Listings from Ghidra, IDA, or similar tools, applied to binaries the analyst lawfully possesses	Algorithm identification, field semantics
Test oracles	Output of the proprietary application for given inputs	Conformance testing

Decompiled output is the most legally sensitive class. Responsible projects limit its use to extracting facts (a field offset, a checksum algorithm, a blend formula) and discard the expression (variable structure, code organization), as discussed below.

2.3 Clean-room workflow

The clean-room methodology predates LLMs. It separates people who observe the original system from people who write the new one, with a written specification as the only channel between them. LLMs can fit into that structure in two ways, and the choice of design determines how defensible the result is.

Design A: Model as specification author. The analysis stage uses a model to convert observations into a prose and schema specification. The specification is reviewed by a human to confirm it describes behavior and contains no copied code. A second session, with no access to the original artifacts, receives only the specification and writes the implementation. The two sessions share no context.

```
[Observation inputs]
        |
        v
[Analysis session]
        |
        v
[Reviewed specification]
        |
        v
[Implementation session]  (no access to inputs)
        |
        v
[Open-source code]
```

Design B: Model as test generator. The model produces test cases from observed behavior (input file, expected output pixels or text). The implementation is written independently and validated against these oracles. This design carries the lowest copying risk because the tests encode behavior only.

A single long session that ingests decompiled code and then writes the replacement defeats the purpose of the separation. Contamination of this kind is the principal process risk when LLMs are used, since a model may reproduce structures it has just read. Mitigations include session isolation, specification review, similarity scanning of generated code against known sources, and provenance logs that record which inputs were available to which session.

2.4 Verification

Model output about binary structures is probabilistic. Reliable projects treat it as a hypothesis generator and validate with deterministic checks:

- Round-trip tests: parse a file, serialize it, and compare bytes.
- Differential testing: render the same document in the reference application and the new implementation, then compare using a perceptual metric such as SSIM or a per-channel delta threshold.
- Fuzzing the parser with mutated inputs to confirm that malformed files are rejected without memory-safety failures.

- Corpus tests across files produced by many versions of the original software.

3. Integrating AI via the Model Context Protocol

3.1 Definition

The Model Context Protocol (MCP) is an open protocol, introduced by Anthropic in November 2024, that standardizes how LLM applications connect to external data sources and tools. It addresses an integration problem: without a shared protocol, each combination of model application and external system requires a bespoke connector, yielding an N×M integration matrix. MCP reduces it to N+M by defining one interface that any compliant client and server can use. The protocol is built on JSON-RPC 2.0 and was designed with ideas from the Language Server Protocol (LSP), which solved a comparable problem for editors and language tooling.

3.2 Architecture

MCP defines three roles:

- **Host**: the application the user interacts with (a chat client, an IDE, a command-line coding agent). The host manages client instances, enforces user consent, and aggregates context.
- **Client**: a connector inside the host that maintains a one-to-one stateful session with a single server.
- **Server**: a process that exposes capabilities to clients.

Transports include stdio, where the host spawns the server as a subprocess and exchanges messages over standard streams, and an HTTP-based transport for remote servers. Sessions begin with a capability negotiation in which client and server declare which features they support.

Servers expose three primitives:

Primitive	Control model	Function
Resources	Application-controlled	Read-only data addressed by URI: files, database rows, logs, documentation pages
Tools	Model-controlled	Functions with JSON Schema–typed inputs that the model may invoke: run tests, query a symbol index, apply a patch
Prompts	User-controlled	Parameterized templates that users select explicitly, such as a code-review workflow

Clients may expose features back to servers, including **roots** (the filesystem boundaries a server may operate within), **sampling** (a server's request for a model completion through the client), and **elicitation** (a server's request for additional input from the user). Because capability sets evolve between protocol revisions, implementers should consult the current specification at modelcontextprotocol.io.

3.3 Embedding MCP in an open-source project workflow

In a static assistant workflow, a developer pastes code into a chat window, receives a suggestion, and manually applies it. Context is stale the moment it is pasted, and the model cannot verify its own output. MCP changes this by letting the model query and act on live project state through typed interfaces. Four architectural patterns recur.

Pattern 1: Codebase intelligence server. A server indexes the repository and exposes tools such as `search_symbols`, `find_references`, `get_call_graph`, and `read_file_range`. Implementations commonly combine tree-sitter parsing for syntax-aware chunking with a language-server backend for type-resolved references. The model navigates a large codebase by targeted queries instead of loading it whole, which keeps the context window available for reasoning.

Pattern 2: Format and specification server. For reverse-engineering projects, a server wraps the project's specification store. Resources expose schema files and annotated sample hex dumps; tools expose `parse_with_schema(file, schema)` and `diff_parse_trees(a, b)`. The model proposes a schema revision, calls the parser, inspects the failure offset, and refines. This loop converts format analysis from conversation into an executable feedback cycle.

Pattern 3: Execution and observability server. A server runs builds and test suites inside a sandbox (a container or a restricted subprocess) and returns structured results: exit codes, failing test names, stack traces, and rendered image artifacts. Logs are exposed as resources, so the model reads the actual failure output instead of a developer's paraphrase. For graphics work, the server can return a rendered frame and a pixel-difference map against the reference oracle.

Pattern 4: Repository operations server. A server wraps the GitHub API (or the `git` CLI) to expose issue reading, branch creation, commit, and pull-request opening. Write-capable tools should be scoped narrowly and gated by human approval for destructive actions such as force-push or merge.

A representative minimal server, written against the TypeScript SDK, illustrates the shape of a tool definition:

```
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from
"@modelcontextprotocol/sdk/server/stdio.js";
import { z } from "zod";

const server = new McpServer({ name: "format-lab", version:
"0.1.0" });

server.tool(
  "parse_with_schema",
  { filePath: z.string(), schemaId: z.string() },
  async ({ filePath, schemaId }) => {
    const result = await runParser(filePath, schemaId); // project-
specific
    return { content: [{ type: "text", text: JSON.stringify(result) }]
};
  }
);
```

```
await server.connect(new StdioServerTransport());
```

3.4 Security considerations

Giving a model tool access creates an attack surface. Relevant controls include least-privilege roots, read-only defaults, explicit user approval for write and network tools, sandboxed execution with no ambient credentials, and treatment of all retrieved content as untrusted input. The last point addresses prompt injection: a sample file or issue comment may contain text that attempts to redirect the model. Servers that process third-party artifacts should return data in clearly delimited fields, and hosts should not grant tool authority on the basis of instructions found inside retrieved content.

3.5 From assistant to development partner

The practical difference is a closed loop. The model reads the failing test through a resource, locates the responsible function through the symbol server, edits the file, runs the test through the execution server, and opens a pull request through the repository server, with the developer reviewing at defined checkpoints. The unit of interaction changes from a single completion to a task, and the human role becomes specification, review, and approval.

4. Methodology: Reverse Engineering Complex Applications

The process below describes interface-level and format-level analysis, the form of reverse engineering generally recognized as legitimate for interoperability. It proceeds in three phases, each producing written artifacts that feed the next.

4.1 Observation and interface analysis

UI and interaction study. Analysts record the application's behavior without inspecting its internals: panel hierarchy, tool options, modal states, undo granularity, and keyboard shortcuts. Screenshot sequences and screen recordings can be passed to a multimodal model to produce a structured inventory of widgets and states. The output is a behavioral catalog, for example: "Selecting the Marquee tool exposes feather, anti-alias, and style options; Shift constrains to a square."

Document model inference. From observed behavior, analysts infer the in-memory document model: a layer tree with groups, per-layer blend mode and opacity, masks, adjustment layers, and smart-object references. This model determines the data structures of the replacement.

Network and IPC observation. Where an application communicates with local helper processes or cloud services, traffic can be observed with standard tools (a local proxy for HTTP, `strace` or Event Tracing for Windows for system calls, platform IPC monitors for pipes and sockets). Analysis is bounded to understanding documented or interoperability-relevant interfaces: for example, how a desktop application exchanges documents with a local plugin host. Observing traffic to learn an authentication scheme in order to access services without authorization is outside the scope of legitimate interoperability work.

4.2 Format and protocol analysis

Container structure. Many proprietary formats are containers. DOCX, XLSX, and PPTX are ZIP archives holding XML parts governed by ECMA-376 and ISO/IEC 29500, which are published standards. PSD is a documented-in-part binary format with sections for header, color mode data, image resources, layer and mask information, and merged image data, using big-endian integers and PackBits or ZIP compression for channel data. Initial analysis identifies magic numbers, section boundaries, and length-prefix conventions.

Iterative schema recovery. The workflow combines three sources: public specifications, differential sampling, and model-assisted hypothesis generation. In differential sampling, the analyst produces file pairs that differ by one controlled change (a layer renamed, opacity set from 100 to 50 percent) and diffs them byte by byte. The model receives the diff, the surrounding structure, and any known specification, then proposes field semantics. Each hypothesis is encoded in a declarative schema (Kaitai Struct, Construct, or a Rust `binrw` definition) and tested against a corpus.

Semantics versus syntax. Parsing bytes is a smaller problem than reproducing meaning. Blend-mode formulas, color-management behavior (ICC profile handling, rendering intents), text layout (kerning, line breaking, font fallback), and filter kernels must match numerically to produce faithful output. These are specified through black-box measurement: feeding known inputs and fitting or confirming a formula against outputs.

Plugin architectures. Extension systems are studied through their public SDKs. Adobe's UXP and CEP frameworks and Microsoft's Office Add-ins model (Office.js, with a web-based runtime) are documented. A replacement can offer a compatible API subset, enabling reuse of some existing extensions, or it can define its own surface.

4.3 Modular re-implementation

The replacement architecture is a design choice, not a copy. Common decisions in current projects:

Layer	Typical choices	Rationale
Core engine (document model, compositing, codecs)	Rust or C++	Memory control, SIMD, predictable performance
GPU rendering	wgpu (WebGPU/Vulkan/Metal/DX12), WebGL for browser targets	Portable GPU access
UI shell	TypeScript with a web framework, hosted in Tauri or Electron; or native toolkits (Qt, GTK)	Developer availability, cross-platform reach
Parsers	Standalone crates or libraries with schema-driven definitions	Reuse across applications, independent testing
Extension runtime	WebAssembly (WASI) sandbox; scripting in TypeScript or Lua	Isolation, language neutrality
Interchange	Open formats (ORA, SVG, ODF, OOXML read/write)	User data portability

A plugin-oriented structure mirrors the modularity of enterprise suites: a small kernel provides the document model and an event bus, and features (filters, tools, import/export codecs) register through a versioned interface. WebAssembly components allow third parties to ship tools in any language that

compiles to it, with capability-based access to the document and host resources. A stable, versioned ABI at the plugin boundary is the main long-term maintenance commitment.

4.4 Role of the LLM across phases

Phase	LLM contribution	Required human control
Observation	Inventory of UI states from recordings; draft behavioral catalog	Verification against the live application
Format analysis	Hypotheses on field semantics; schema drafts; test-vector generation	Corpus validation, round-trip tests
Implementation	Module code drafts, refactoring, test authoring	Code review, performance profiling, provenance audit

4.5 Case study: ArtCraft

ArtCraft is a family of free, open-source desktop and web applications published under the GitHub account `storytold`, with a project site at getartcraft.com and a Discord community. The project calls its applications “Crafting Apps” and states that each is written from scratch in Rust and can stand on its own. The repositories describe themselves as clean-room reimplementations of commercial products. The table summarizes the applications reviewed for this paper; descriptions are taken from each repository’s public page, and star counts are a snapshot from October 2026.

Application	Commercial reference stated by the project	Stated scope
PhotoCraft	Not stated in the pages reviewed	Image editing: layers, masks, type, PSD files
VectorCraft	Not stated in the pages reviewed	Vector illustration
FilmCraft	Not stated in the pages reviewed	Video editing, color, and sound
CADCraft	Autodesk AutoCAD	Command line, object snaps, layers, dimensions, hatches, blocks, DXF/DWG
LightCraft	Adobe Lightroom	Photo library and raw development; 32-bit float, wide-gamut pipeline; 3.6k stars
PdfCraft	Adobe Acrobat	Read, organize, combine, split, and secure PDFs; 4.6k stars
FigCraft	Figma	Frames, auto layout, components, variants, vector networks, prototypes; 802 stars
EffectCraft	Adobe After Effects	Compositions, keyframes, 306 effects, expressions, 3D cameras, render queue
SoundCraft	Avid Pro Tools	Multitrack DAW with mixer, MIDI, recording, and automation; pre-alpha; 509 stars

Four observations connect the project to the themes of this paper.

Stack and portability. All applications are written in Rust. Several target macOS, Windows, and Linux natively and run in the browser through WebAssembly; SoundCraft also lists FreeBSD and uses the egui toolkit. This matches the core-engine and GPU-portability choices in Section 4.3.

Format interoperability. CADCraft reads and writes DXF natively (ASCII and binary, R12 through 2018) and opens DWG files (R13 through 2018) through the open-source acadrust library. FigCraft opens `.fig` files on a best-effort basis and accepts designs exported as public REST API file JSON. PhotoCraft states support for real PSD files. PdfCraft opens protected documents and states that it respects their usage rules. These are examples of the schema-recovery and public-specification approach in Section 4.2, including the practice of reusing an existing open-source parser instead of writing a new one.

Agent integration. LightCraft states that it can be driven end to end by AI agents over MCP and ships an MCP server. SoundCraft states that every action is scriptable from the command line or by an agent, that programmatic calls never open dialogs, and that a `session.inspect` call reports exactly what changed so an agent can check its own work. This is the closed verification loop of Section 3.5 built into the application: the program exposes typed operations and a way to read back state, so an agent does not depend on screen scraping.

Licensing and governance. Several repositories use the dual license MIT OR Apache-2.0. The ArtCraft name and logos are trademarks of the ArtCraft Team and are excluded from the open-source license, which illustrates the separation between code license and trademark discussed in Section 5.3. Repositories carry maturity labels such as “young and moving fast” and “pre-alpha.”

Limits of the evidence. The public pages reviewed do not describe whether or how LLMs were used to develop the applications, so this paper makes no claim about the project’s development workflow. Clean-room status is self-reported, and no provenance audit or fidelity benchmark against the commercial products was found. The pages also do not state whether the applications share a common engine. These questions are suitable for follow-up work using repository history, contribution policies, and conformance testing.

5. The Ecosystem Catalyst: GitHub’s Role

5.1 Coordination layer for distributed development

GitHub provides identity, hosting, issue tracking, review, automation, and distribution under one account system. For decentralized projects with no employer relationship among contributors, this integration lowers the coordination cost of accepting contributions from strangers. The fork-and-pull-request model allows any person to propose changes without prior access, and the repository’s maintainers hold merge authority. Issues, Discussions, and Projects serve as the shared backlog for specifications, format findings, and compatibility reports.

For reverse-engineering projects specifically, the repository often holds the specification as a first-class artifact: schema files, annotated format documents, and conformance corpora versioned alongside code. This makes the specification reviewable through the same pull-request process as the implementation, which supports the clean-room audit trail described in Section 2.3.

5.2 Pull requests, CI/CD, and AI agents

Automation through GitHub Actions turns repository events into workflows. In AI-assisted projects the following pipeline is common:

1. A contributor, or an agent acting on a contributor's behalf, opens a pull request.
2. CI builds the project across a matrix of operating systems and toolchain versions.
3. Unit, round-trip, and differential rendering tests run; image diffs are attached as artifacts.
4. Static analysis (clippy, clang-tidy, CodeQL), dependency auditing, and license scanning run.
5. A similarity check compares new code against known third-party sources.
6. An AI reviewer comments on the diff, identifying defects and style violations.
7. A human maintainer reviews and merges after required checks pass, enforced through branch protection rules.

AI agents interact with this pipeline in two ways. Agents driven by tools such as Claude Code or GitHub-integrated assistants can be assigned an issue, create a branch, push commits, and respond to CI failures by iterating until checks pass. Separately, repository workflows can invoke a model during CI to triage failures, summarize logs, or draft release notes. In both cases, the CI suite functions as the objective acceptance test. Its quality bounds the safety of AI-generated contributions: a project with a weak test corpus will accept plausible but incorrect code, while a project with strong differential tests can absorb high volumes of machine-written changes.

Recommended controls for AI-originated changes include mandatory human review for merges, disclosure of AI assistance in the pull-request description or commit trailers, restricted workflow permissions (read-only `GITHUB_TOKEN` by default), and protection against workflow-level injection from untrusted pull-request content.

5.3 Governance of community-driven clones

Projects of this type face governance questions that single-vendor products do not.

Licensing. Copyleft licenses (GPL, AGPL) keep derivatives open, and permissive licenses (MIT, Apache-2.0) maximize adoption and allow proprietary forks. Apache-2.0 additionally provides an express patent grant, which is relevant when the subject domain is dense with patents. A Developer Certificate of Origin (DCO) sign-off or a contributor license agreement (CLA) records that contributors have the right to submit their work.

Provenance policy. Mature projects publish a contribution policy that prohibits submitting code derived from decompiled proprietary sources, requires that specifications describe behavior only, and defines how AI-assisted contributions are disclosed. Enforcing the policy requires reviewers who understand what contamination looks like.

Trademark and naming. Projects avoid names and iconography that suggest affiliation with the original vendor. Trademark exposure is distinct from copyright exposure and is often the more immediate source of takedown requests.

Stewardship. Sustained maintenance of a rendering engine requires funding and continuity. Fiscal hosts, foundations, and sponsorship platforms provide the legal entity and funds, and a documented succession and decision-making process (maintainer council, RFC process) reduces dependence on individuals.

6. Challenges, Ethical Considerations, and Future Outlook

6.1 Technical challenges

Performance. Professional editors embody decades of optimization: tiled image pipelines, SIMD-vectorized kernels, GPU-resident compositing, and incremental invalidation. Matching latency on large documents (multi-gigapixel images, spreadsheets with millions of rows) is hard to achieve by translation from specification alone. LLM-generated code tends toward correctness-first implementations, so profiling-driven optimization remains skilled human work.

Fidelity of rendering and layout. Pixel-exact agreement is rarely achievable, and perceptual equivalence thresholds must be negotiated. Text shaping and layout across scripts is a substantial subproblem; projects usually adopt existing libraries (HarfBuzz, FreeType) instead of reimplementing.

Format long tail. Real-world files contain vendor quirks, version-specific behavior, and malformed-but-accepted structures. Compatibility accumulates through years of bug reports. A model can accelerate fixes once a failing sample exists, but it cannot discover samples that nobody has contributed.

Legacy and ecosystem compatibility. Users depend on third-party plugins, scripts, templates, and color profiles. Compatibility layers for scripting APIs (for example, VBA in Office) are large and legally and technically complex.

Maintenance of AI-generated code. Code produced at high volume can outpace maintainers' ability to understand it. Architecture documentation, modular boundaries, and strong test suites are the primary mitigation.

Model reliability. Hallucinated field semantics, plausible but wrong formulas, and silent regression introduced during refactoring are documented failure modes. Verification infrastructure, not model quality alone, determines project outcomes.

6.2 Legal and ethical considerations

This section summarizes principles and does not constitute legal advice; outcomes depend on jurisdiction and facts.

Copyright protects expression, not function. In U.S. law, copyright does not extend to ideas, procedures, or methods of operation (17 U.S.C. § 102(b); *Baker v. Selden*). Courts have held that disassembly to access unprotected elements can be fair use where necessary and no other means exists (*Sega v. Accolade*, 9th Cir. 1992; *Sony v. Connectix*, 9th Cir. 2000). In *Google v. Oracle* (2021), the Supreme Court found that Google's reuse of declaring code from the Java API was fair use, without resolving whether such code is copyrightable. These decisions support functional re-implementation but do not authorize copying of code, assets, or creative content.

Statutory interoperability provisions. The DMCA, 17 U.S.C. § 1201(f), permits circumvention for the limited purpose of achieving interoperability of an independently created program, subject to conditions. The EU Software Directive (2009/24/EC), Article 6, permits decompilation where indispensable to obtain interoperability information, and bars use of that information to develop a substantially similar program that infringes expression. Both provisions are narrow, and neither covers copying code.

Contractual restrictions. End-user license agreements frequently prohibit reverse engineering. Their enforceability varies by jurisdiction and by whether the analyst accepted the license. Projects that rely on public documentation and black-box observation reduce this exposure.

Patents and trademarks. Clean-room design offers no defense to patent infringement, and algorithms used in codecs, color processing, and fonts can be patent-encumbered. Trademark law constrains naming and visual identity independently of copyright.

Assets and data. Fonts, icon sets, brushes, templates, ICC profiles, and sample content bundled with commercial software carry separate licenses. They cannot be redistributed with an open-source replacement.

LLM-specific risks. Models may reproduce memorized training data. If an analyst exposes a model to proprietary code and subsequently asks it to generate an implementation, a plaintiff could argue derivation. Session isolation, review of generated output, and documented provenance are the practical safeguards. The legal status of AI-generated code, including authorship and training-data questions, remains unsettled in several jurisdictions.

Ethical considerations. Interoperability work serves legitimate interests: document preservation, accessibility for users who cannot afford licenses, user ownership of data, and competition. The same techniques can be applied to circumvent licensing or to appropriate another party's work, and the distinction rests on whether the project recreates function from lawful observation or takes protected expression. Contributors also carry responsibility for transparent disclosure of AI use and for honoring the license terms of any dependency.

6.3 Future outlook

Several trends follow from the analysis above.

1. **Specification becomes the primary artifact.** If implementation cost declines, the durable asset is a verified, machine-readable behavioral specification with a conformance corpus. Projects that invest in specifications and oracles can regenerate or port implementations as platforms change.
2. **Verification infrastructure determines viability.** The limiting factor shifts from writing code to establishing trustworthy tests. Differential testing against reference applications, and shared public conformance suites for formats such as OOXML and PSD, become shared infrastructure.
3. **Protocol standardization of AI tooling.** MCP-style interfaces allow project-specific servers (format labs, render oracles) to be reused across model providers and editors, which lowers switching costs and makes tooling a community-maintained asset.
4. **Pressure on proprietary advantage.** Feature parity for common workflows becomes cheaper to reach, which erodes lock-in based on format opacity and interface familiarity. The remaining sources of advantage are performance, ecosystem depth, service integration, support, and trust. Vendors are likely to respond with differentiated cloud and AI features, stricter terms, and use of trademark and patent claims, and legal challenges to AI-assisted re-implementation are probable.
5. **Governance and provenance standards.** Expect formal practices for recording AI involvement, scanning for derivation, and certifying clean-room processes, similar to the way software bills of materials have become standard for dependency tracking.

Proprietary suites are unlikely to be displaced by direct feature cloning alone. Open alternatives have historically succeeded where they reached a sufficient quality threshold for a defined user segment and offered properties the incumbent could not, such as open formats, local-first operation, scriptability, or no licensing cost. AI-assisted analysis shortens the path to the threshold; it does not remove the need for sustained maintenance, design judgment, and community governance.

7. Conclusion

LLMs reduce the cost of understanding existing systems, MCP gives them structured and permissioned access to the artifacts and tools that understanding requires, and GitHub supplies the coordination and automation layer in which the results are reviewed and merged. The resulting workflow moves engineering effort toward specification, verification, and governance. Its technical outcomes depend on test infrastructure more than on model capability, and its legitimacy depends on adherence to clean-room discipline, which separates observed function from protected expression. Future work should quantify defect rates in AI-derived format parsers, evaluate provenance-tracking methods, and develop shared conformance corpora for widely used proprietary formats.

References and Further Reading

- Model Context Protocol specification and SDK documentation, modelcontextprotocol.io.
- ECMA-376 / ISO/IEC 29500, Office Open XML File Formats.
- Adobe Photoshop File Formats Specification (Adobe public documentation).
- *Sega Enterprises Ltd. v. Accolade, Inc.*, 977 F.2d 1510 (9th Cir. 1992).
- *Sony Computer Entertainment, Inc. v. Connectix Corp.*, 203 F.3d 596 (9th Cir. 2000).
- *Google LLC v. Oracle America, Inc.*, 593 U.S. 1 (2021).
- 17 U.S.C. §§ 102(b), 1201(f).
- Directive 2009/24/EC of the European Parliament and of the Council on the legal protection of computer programs, Article 6.
- Language Server Protocol specification, Microsoft.
- Kaitai Struct documentation, kaitai.io.

ArtCraft case study sources, accessed October 16, 2026: CADCraft, LightCraft, PdfCraft, FigCraft, EffectCraft, SoundCraft. Details should be rechecked against the live repositories, since versions and star counts change.